GUÍA Y APUNTES DE PROGRAMACIÓN EN C++

--Kerem Akcan--

Estos apuntes están diseñados para aprender C++ desde cero. Todo se explica paso a paso con códigos sencillos para que un estudiante principiante pueda entender fácilmente la lógica de programación.

MÓDULO 1: Fundamentos Esenciales de C++

1.1. Entrada y Salida de Datos

Todo programa en C++ necesita la librería <iostream> para mostrar y leer datos. Usamos 'cout' para escribir en la pantalla y 'cin' para guardar lo que escribe el usuario.

```
#include <iostream> // Libreria obligatoria para mostrar y leer datos
using namespace std; // Evita escribir std:: en cada linea

int main() { // Punto de inicio del programa
    int numero; // 'int' declara una variable de tipo entero
    cout << "Introduzca un numero: "; // Imprime el texto en pantalla
    cin >> numero; // Lee lo ingresado por teclado y lo guarda en 'numero'
    cout << "El doble es: " << (numero * 2) << endl; // 'endl' hace un salto de
linea
    return 0; // Indica que el programa finalizo con exito
}
```

1.2. Decisiones (If - Else)

Usamos 'if' y 'else' para tomar decisiones. El programa comprueba una condición y hace una tarea diferente dependiendo del resultado.

```
#include <iostream>
using namespace std;

int main() {
    double nota; // 'double' permite guardar numeros con decimales
    cout << "Ingrese la nota final: ";
    cin >> nota;

    if (nota >= 5.0) { // Si (if) la nota es mayor o igual (>=) a 5.0
        cout << "Has aprobado." << endl;
    } else { // En caso contrario (else), si no se cumple la condicion
        cout << "Has suspendido." << endl;
    }
    return 0;
}
```

1.3. Bucles (For, While, Do-While)

Los bucles repiten el código. El bucle 'do-while' es perfecto para crear menús interactivos porque se ejecuta al menos una vez.

```
#include <iostream>
using namespace std;

int main() {
    int opcion;
    do { // Ejecuta este bloque al menos una vez (hacer...)
        cout << "1. Continuar" << endl;
        cout << "2. Salir" << endl;
        cout << "Elige una opcion: ";
        cin >> opcion;
    } while (opcion != 1 && opcion != 2); // ...mientras sea distinto (!=) de 1 Y
(&&) distinto de 2

    cout << "Opcion correcta. Saliendo..." << endl;
    return 0;
}
```

1.4. Listas (Arrays Estáticos)

Un array (lista) sirve para guardar muchos datos del mismo tipo en una sola variable. Los índices empiezan en 0.

```
#include <iostream>
using namespace std;

int main() {
    int inventario[5] = {10, 20, 30, 40, 50}; // Crea un array de 5 espacios con
datos iniciales
    int suma = 0; // Inicializamos a cero antes de sumar

    // Bucle for: 'i' empieza en 0; continua si 'i < 5'; y suma 1 ('i++') en cada
paso
    for (int i = 0; i < 5; i++) {
        suma += inventario[i]; // '+=' anade el valor del inventario a la suma
total
    }

    cout << "La suma total es: " << suma << endl;
    return 0;
}
```

MÓDULO 2: Algoritmos Fundamentales y Funciones

2.1. Funciones y Cálculo Factorial

Las funciones nos permiten separar el código. En este ejemplo, calculamos el factorial de un número multiplicando desde el 1.

```
#include <iostream>
using namespace std;

// Funcion que recibe un numero entero (n) y devuelve otro entero
int calcularFactorial(int n) {
    int resultado = 1;
    for (int i = 1; i <= n; i++) {
        resultado *= i; // '*=' multiplica el resultado actual por 'i'
    }
    return resultado; // Devuelve el calculo final al bloque principal
}

int main() {
    int numero = 5;
    cout << "El factorial de 5 es: " << calcularFactorial(numero) << endl;
    return 0;
}
```

2.2. Algoritmo para Números Primos

Un número es primo si solo se puede dividir entre 1 y sí mismo. Comprobamos si tiene divisores usando un bucle.

```
#include <iostream>
using namespace std;

// 'bool' es un tipo de dato que solo puede ser verdadero (true) o falso (false)
bool esPrimo(int numero) {
    if (numero <= 1) return false;

    for (int i = 2; i < numero; i++) {
        if (numero % i == 0) { // El operador '%' calcula el resto de la division
            return false;
        }
    }
    return true; // Si no ha tenido ningun divisor extra, es primo
}

int main() {
    if (esPrimo(7)) {
        cout << "Es un numero primo." << endl;
    } else {
        cout << "No es primo." << endl;
    }
    return 0;
}
```

MÓDULO 3: Algoritmos con Listas

3.1. Serie de Fibonacci

La secuencia de Fibonacci suma los dos números anteriores. Aquí llenamos un array pasando los datos a una función.

```
#include <iostream>
using namespace std;

// 'void' significa que esta funcion hace una tarea, pero no devuelve ningun
valor
void hacerFibonacci(int n, int lista[]) { // 'lista[]' permite recibir y
modificar el array original
    if (n >= 1) lista[0] = 0;
    if (n >= 2) lista[1] = 1;

    for (int i = 2; i < n; i++) {
        lista[i] = lista[i - 1] + lista[i - 2];
    }
}

int main() {
    int secuencia[5]; // Declaramos una lista vacia de 5 espacios
    hacerFibonacci(5, secuencia); // Pasamos la lista a la funcion para que la
rellene

    cout << "Fibonacci: ";
    for (int i = 0; i < 5; i++) {
        cout << secuencia[i] << " ";
    }
    cout << endl;
    return 0;
}
```

3.2. Ordenar una Lista (Bubble Sort)

El método burbuja compara dos números vecinos y los cambia de lugar si el primero es mayor que el segundo.

```
#include <iostream>
using namespace std;

void ordenar(int lista[], int tamano) {
    for (int i = 0; i < tamano - 1; i++) {
        for (int j = 0; j < tamano - 1 - i; j++) {
            if (lista[j] > lista[j + 1]) {
                // Usamos una variable temporal 'aux' para no borrar el dato al
intercambiar
                int aux = lista[j];
                lista[j] = lista[j + 1];
                lista[j + 1] = aux;
            }
        }
    }
}

int main() {
    int datos[5] = {5, 1, 4, 2, 8};
    ordenar(datos, 5);

    for (int i = 0; i < 5; i++) {
        cout << datos[i] << " ";
    }
    return 0;
}
```

MÓDULO 4: Pruebas Automáticas

Podemos usar 'assert' para comprobar que el código funciona automáticamente. Si algo está mal, el programa se detiene.

```
#include <iostream>
#include <cassert> // Libreria obligatoria para hacer pruebas (assert)
using namespace std;

int sumar(int a, int b) {
    return a + b;
}

int main() {
    // 'assert' revisa que la afirmacion sea verdad. Si es mentira, el programa
se cierra con error
    assert(sumar(2, 3) == 5);
    assert(sumar(0, 0) == 0);

    cout << "va bien" << endl;
    return 0;
}
```